

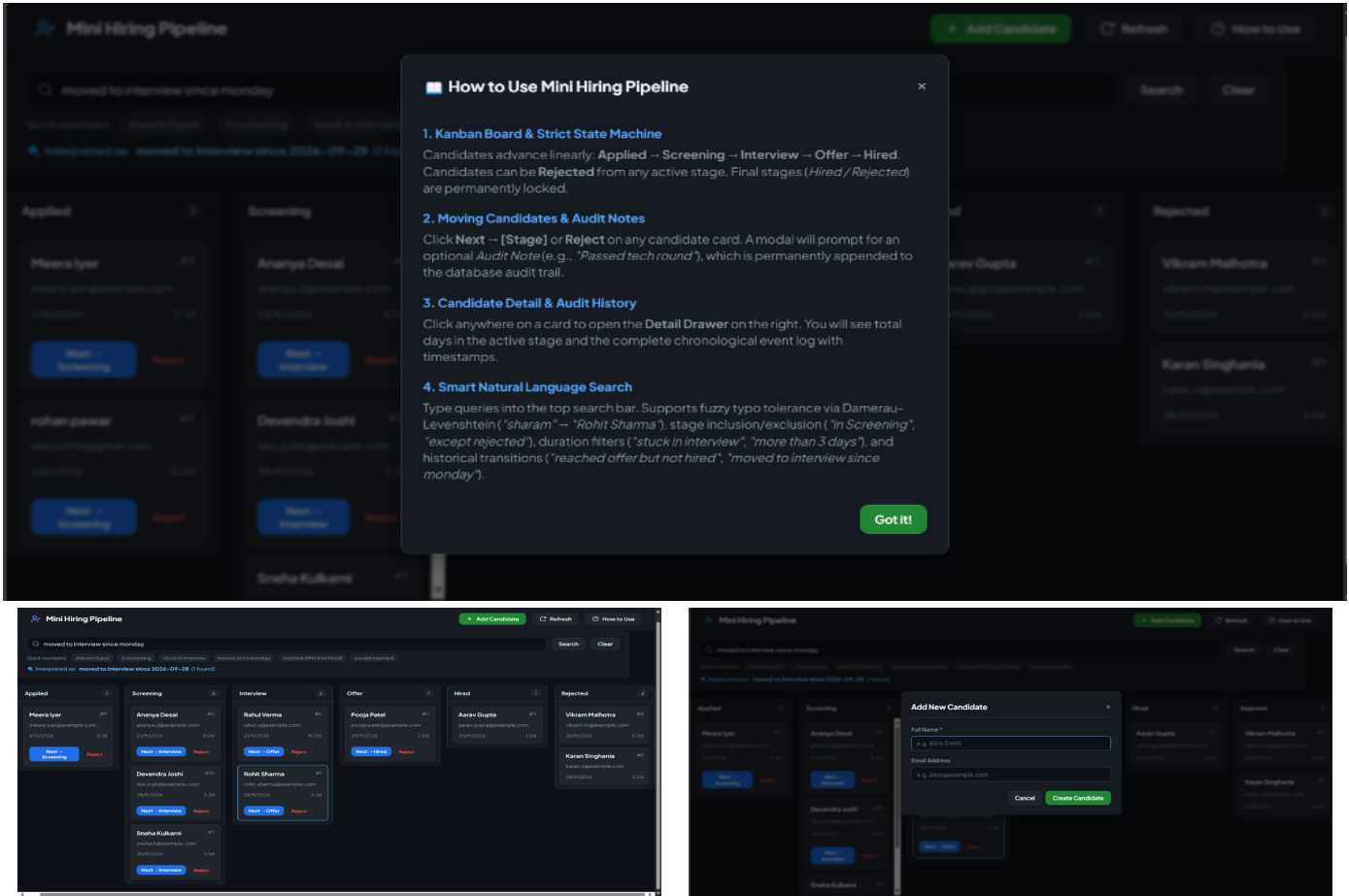
Mini Hiring Pipeline — Architecture & Decisions

Deliverable Documentation, System Flow & Technical Overview

GitHub: <https://github.com/RohitPawar001/hiring-pipeline> | Stack: FastAPI, PostgreSQL 16 (Event Sourced), Docker Compose, Vanilla Web UI

1. User Interface & Interactive Workflow

A real-time recruiter board with smart search, stage duration tracking, audit history timeline drawers, and strict state machine transition controls:

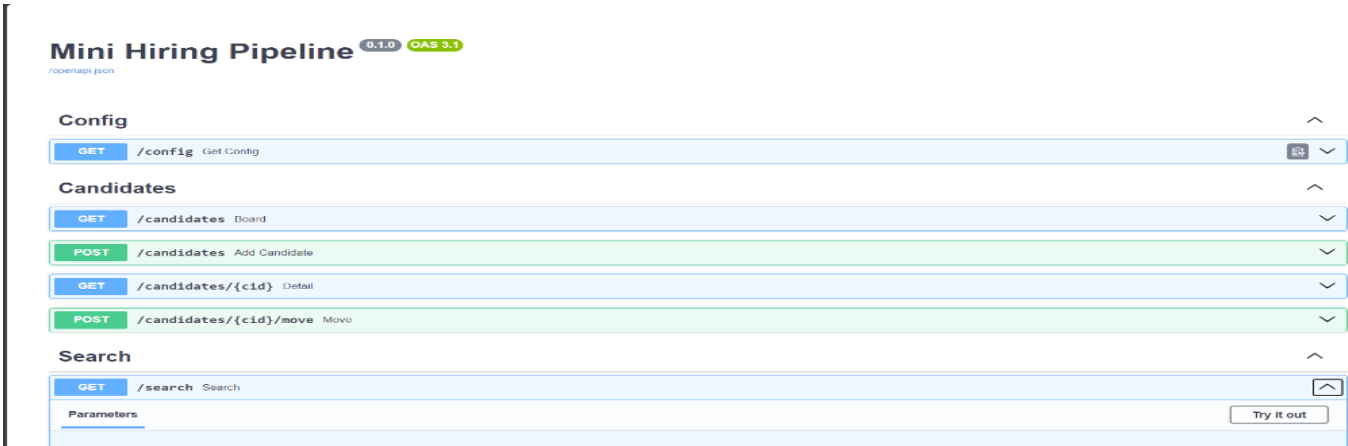


2. System Architecture & Flow Diagram

Layer	Component & Responsibility	Data / State Interaction
Frontend (UI)	Kanban Board, NL Search Box, Audit Timeline Drawer, Add & Move Modals (Vanilla JS/CSS)	Fetches GET /config on init. Issues REST calls for moves & query filters.
API Layer	FastAPI Modular Routers: /candidates, /search, /config	Validates schemas with Pydantic; manages transactions & row-level locks.
State Machine	Strict Transition Engine: Applied → Screening → Interview → Offer → Hired	Validates sequential forward steps and permits Rejected; locks terminal states.
Search Parser	Rule-based tokenizer + Damerau-Levenshtein transposition distance matrix	Translates natural queries into SQL predicates and scores typo matches (e.g. sharam).
PostgreSQL 16	Append-Only stage_events + Derived candidate_current view	PL/pgSQL triggers raise exceptions on UPDATE/DELETE/TRUNCATE for tamper-proof history.

3. Modular API Design & OpenAPI / Swagger Specs

FastAPI provides automated, interactive Swagger documentation with strong Pydantic typing and validation schemas:



4. Where I Disagreed with the AI

- **Test Cleanup via TRUNCATE CASCADE:** The AI initially suggested cleaning test data using TRUNCATE candidates CASCADE. I disagreed because cascading truncates fire child triggers on stage_events, causing tests to crash on our own immutability rules. Instead, I designed test fixtures using UUID-isolated candidate records.
- **Standard Levenshtein vs Damerau-Levenshtein:** The AI proposed standard Levenshtein distance for typos. I rejected this because transpositions (like *sharam*) counted as 2 edits, failing the threshold. Implementing full Damerau-Levenshtein matrix evaluation correctly identified adjacent transpositions as a single edit.

5. Deliverables Checklist & Future Improvements

- **Deliverables Included:** Architecture PDF, Complete GitHub Readme with setup & decisions, Full AI chat conversation transcripts under chat_logs/, and Comprehensive Pytest suite.
- **Future Scale:** PostgreSQL pg_trgm GiST indexing for >100k candidates, Multi-tenant Recruiter JWT authentication with actor logging, and WebSocket live updates.